

```

#include <ESP32Servo.h>
#include <ArduinoJson.h>

// Objets servo :
// - esc = contrôleur moteur (ESC)
// - servo = direction
Servo esc;
Servo servo;

// ÉTAT ACTUEL
// Valeurs actuelles envoyées (en microsecondes)
int currentServoUS = 1450;
int targetServoUS = 1450;
int currentSpeedUS = 1500;
int targetSpeedUS = 1500;

// LIMITES SERVO
// Bornes physiques de la direction
const int SERVO_MIN = 1200;
const int SERVO_MAX = 1650;
const int SERVO_NEUTRE = 1450;

// LISSEMENT
// Permet d'éviter les mouvements brusques
const int SMOOTH_STEP = 10; // Incrément à chaque update
const int SMOOTH_DELAY = 120; // Temps entre chaque update (ms)
unsigned long lastSmoothTime = 0;

// COOLDOWN
// Évite de traiter trop de commandes trop rapidement
unsigned long lastCmdTime = 0;
const unsigned long CMD_INTERVAL = 40; // délai minimum entre commandes

// FILTRE ANGLE
// Ignore les petites variations inutiles
int lastAngle = 9999;
const int ANGLE_THRESHOLD = 3;

// WATCHDOG COMMUNICATION
// Sécurité si perte de communication
unsigned long lastCommTime = 0;
const unsigned long COMM_TIMEOUT = 3000; // 3 secondes

void setup() {
    Serial.begin(115200);

    // Attache les pins aux servos
    servo.attach(13);
    esc.attach(14);

    // Initialisation ESC (neutre)
    esc.writeMicroseconds(1500);
    delay(3000); // temps nécessaire pour armer l'ESC

    // Centre la direction
    servo.writeMicroseconds(SERVO_NEUTRE);

    Serial.println("ESP32 READY (MODE F/R)");
}

void updateSmooth() {
    unsigned long now = millis();

    // Respect du timing de lissage

```

```

if (now - lastSmoothTime < SMOOTH_DELAY) return;
lastSmoothTime = now;

// LISSAGE DIRECTION
// Approche progressivement la cible
if (currentServoUS < targetServoUS) currentServoUS += SMOOTH_STEP;
else if (currentServoUS > targetServoUS) currentServoUS -= SMOOTH_STEP;

// Sécurise dans les bornes
currentServoUS = constrain(currentServoUS, SERVO_MIN, SERVO_MAX);

// Envoie au servo
servo.writeMicroseconds(currentServoUS);

// LISSAGE VITESSE
if (currentSpeedUS < targetSpeedUS) currentSpeedUS += SMOOTH_STEP;
else if (currentSpeedUS > targetSpeedUS) currentSpeedUS -= SMOOTH_STEP;

currentSpeedUS = constrain(currentSpeedUS, 1200, 2000);

esc.writeMicroseconds(currentSpeedUS);
}

// FAILSAFE
// Met le système en sécurité si plus de communication
void checkCommTimeout() {
    unsigned long now = millis();

    if (now - lastCommTime > COMM_TIMEOUT) {
        // Communication perdue → retour au neutre
        targetSpeedUS = 1500;
        targetServoUS = SERVO_NEUTRE;

        // Message affiché une seule fois
        static bool alreadyWarned = false;
        if (!alreadyWarned) {
            Serial.println("Timeout communication -> activation mode neutre");
            alreadyWarned = true;
        }
    } else {
        // Reset du flag si communication OK
        static bool alreadyWarned = false;
        alreadyWarned = false;
    }
}

void loop() {
    updateSmooth();           // Applique le lissage
    checkCommTimeout();       // Vérifie la communication

    // Rien à lire → on sort
    if (!Serial.available()) return;

    // Lecture d'une ligne JSON
    String jsonStr = Serial.readStringUntil('\n');
    jsonStr.trim();
    if (jsonStr.length() == 0) return;

    // Mise à jour du watchdog (communication OK)
    lastCommTime = millis();

    // COOLDOWN
    unsigned long now = millis();
    if (now - lastCmdTime < CMD_INTERVAL) return;

```

```

lastCmdTime = now;

// PARSING JSON
StaticJsonDocument<256> doc;
if (deserializeJson(doc, jsonStr)) {
    Serial.println("JSON ERROR");
    return;
}

// Récupération de l'action
String action = doc["action"].as<String>();

// COMMANDE DRIVE
if (action == "drive") {
    int angle = doc["angle"];
    int speed = doc["speed"];

    // Mise à jour vitesse cible
    targetSpeedUS = speed;

    // Normalisation angle [-180 ; 180]
    int a = angle % 360;
    if (a > 180) a -= 360;

    // Filtrage des petites variations
    if (abs(a - lastAngle) >= ANGLE_THRESHOLD) {
        lastAngle = a;

        // Limitation sécurité
        int limited = constrain(a, -180, 180);

        // Conversion angle → microsecondes servo
        targetServoUS = map(limited, -180, 180, SERVO_MAX, SERVO_MIN);
    }

    // Debug
    Serial.print("CMD angle=");
    Serial.print(angle);
    Serial.print(" servoUS=");
    Serial.print(targetServoUS);
    Serial.print(" speed=");
    Serial.print(speed);
    Serial.print(" currentSpeed=");
    Serial.println(currentSpeedUS);
}

// --- COMMANDE STOP ---
else if (action == "stop") {
    targetSpeedUS = 1500; // retour neutre moteur
    Serial.println("STOP CMD");
    delay(1000); // petite pause
}
}

```